

Learn to Design and Verify a 6-stage pipelined RISC-V CPU

Rishiyur S. Nikhil, Cofounder and CTO, Bluespec, Inc.

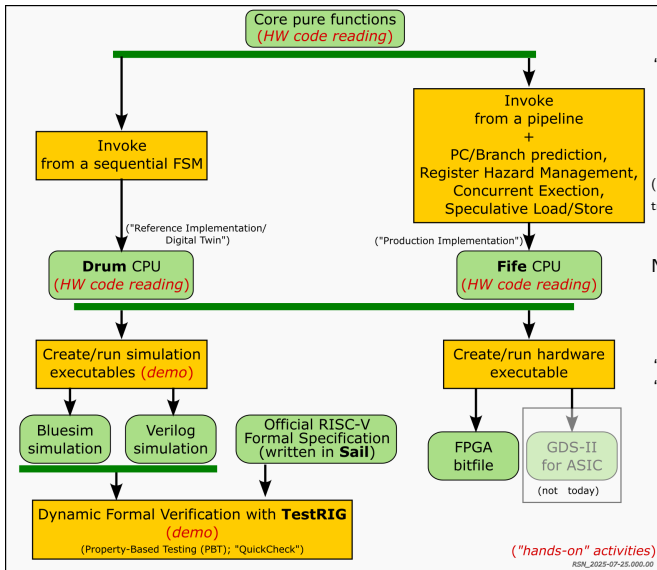
October 22, 2025

Tutorial at Developer's Day, RISC-V Summit, Santa Clara CA



DRAFT: Final version will be available closer to tutorial day

The Plan for Today



"Hands on":

- We will read the code together
- We create and run simulation executables together
- We will verify the design together

(if you haven't installed the tools yet, please just listen and follow; hopefully you will try everything on your own, later)

Not "Hands on" (lecture and demo only):

- How to build and run on FPGA

"But I'm not familiar with hardware design" or
"I'm not familiar with the language used" ...
... don't worry:

- For "Core pure functions" and **Drum**, the code looks just like a conventional programming language
- **Fife** will introduce pipelining and pipeline techniques; we will explain the code in more detail.

Prerequisites

We *do* assume that you are familiar with:

- Some ISA (Instruction Set Architecture): Machine Instructions and/or assembly-language (RISC-V or other).

We're not going to explain what is a general-purpose register, what is a Program Counter, what is an arithmetic instruction, what is a BRANCH or JAL/JALR instruction, what is a LOAD/STORE instruction, etc.; we're assuming you already know these things.

- General coding (in C, Python, Go, Rust, Haskell, Verilog, SystemVerilog, ... whatever), so you can quickly grasp new, unfamiliar code.
Familiarity with enum and struct types, functions, if-then-else.
General knowledge of object-oriented programming is useful.

We *do not* assume that you are familiar with:

- Hardware design, or any hardware design language (Verilog, SystemVerilog, Bluespec BSV or BH, Chisel, ...)
- Pipelined micro-architecture of CPUs

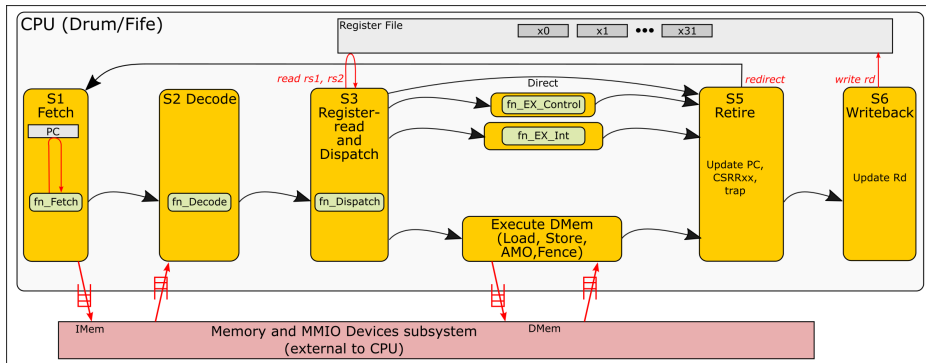
On the Tutorial Web Page (https://github.com/rsnikhil/Tutorial_RISCV_Summit_2025), we've asked you to install certain tools so that you can examine code and build and run things along with me during this tutorial.

If you haven't done this already, then please just sit back and follow the lecture and demos, for now. We hope that you will be motivated to later install the tools and try everything out for real.

Contents

- 1 Core pure functions
- 2 **Drum**
- 3 **Drum**: Generating Verilog. Simulation
- 4 **Fife**
- 5 **Fife**: Generating Verilog. Simulation
- 6 Verification using test programs
- 7 Verification using **TestRIG**
- 8 Execution on FPGA

Basic steps in executing an instruction (any CPU, incl. **Drum** and **Fife**)



This diagram illustrates the core steps in executing any instruction. The “`fn_XXX`” boxes indicate core functions within each step.

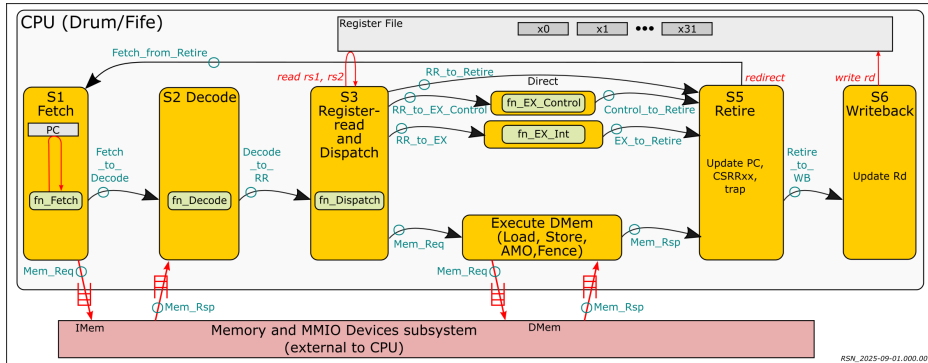
The “Memory and MMIO Devices subsystem” is *external* to the CPU and is not our focus today (we will in fact use C code to “model” this in simulation). The CPU sends requests to memory; for each request and the memory sends a response back to the CPU, *after an unknown latency*. This is an “asynchronous split-phase transaction.”

Memory-access latency can vary dynamically due to possible caching, congestion, varying-length paths to different memory units/devices etc..

The CPU correctness should be robust to this varying latency.

Core pure functions (shared by **Drum** and **Fife**)

Core pure functions (shared by **Drum** and **Fife**)



This diagram annotates each inter-step/stage arrow with the "struct" type it communicates (in green text).

The core functions are pure functions, and are shared by **Drum** and **Fife**:

E.g.,
fn_Fetch: (pure) function from the PC to a pair of structs of type Mem_Req and Fetch_to_Decode
fn_Decode: (pure) function from a pair of structs of type Fetch_to_Decode and Mem_Rsp to a struct of type Decode_to_RR

... and so on.

Let's read some code: Coding of memory request/response types

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/Instr.Bits.bsv
Focus on:	funct3-{LB,LH,LW,LD,SB,SH,SW,SD}, funct5-{LOAD/STORE}

- These are the bit-encodings of fields in RISC-V LOAD and STORE instructions that indicate whether the instruction is a LOAD or a STORE, and whether it is for a Byte (B), Halfword (H), Word (W) or doubleword (D).

These are defined in *The RISC-V Instruction Set Manual Volume I, Unprivileged Architecture*, at <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view> Chapter 35: *RV32/64G Instruction Set Listings*

Other funct5_* definitions are for AMO instructions, distinguishing between FETCH and LOAD, etc.

Let's read some code: struct Mem_Req (memory requests)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/Mem_Req_Rsp.bsv
Focus on:	struct Mem_Req

- In struct Mem_Req, the req_type field has type Mem_Req_Type, which is defined earlier in the file to be a synonym for Bit #(5), i.e., a 5-bit value, corresponding to the funct5_* definitions from the previous slide (LOAD? STORE?).
- The req_size field has type Mem_Req_Size which is defined earlier in the file as an “enum” type indicating whether a memory request is for 1, 2, 4 or 8 bytes.
- The addr and data fields are of type Bit #(64), i.e., 64-bit wide values. The data field is only relevant when the request is for a STORE instruction; it can be ignored for Fetches, LOADs, and LRs.

Note:

We could have used Bit #(XLEN) for these types, where XLEN is defined in file Code/src_Common/Arch.bsv to be either 32 (for RISC-V RV32 implementation) or 64 (for RISC-V RV64 implementation). Decoupling the CPU's XLEN from the memory port width is useful for many reasons:

- A wider memory fetch port allows pre-fetching, and facilitates compressed-instruction parsing.
- A wider memory load/store port may be needed for double-precision floating point in an RV32 CPU.

- The remaining fields of struct Mem_Req are not relevant for the moment. The epoch field is only relevant for **Fife**, and the remaining fields are only for debugging.

Let's read some code: struct Mem_Rsp (memory responses)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/Mem_Req_Rsp.bsv
Focus on:	struct Mem_Rsp

- The field `rsp_type` has type `Mem_Rsp_Type` which is defined earlier in the file to be an `enum` with values `MEM_RSP_OK`, `MEM_RSP_MISALIGNED`, `MEM_RSP_ERR`, or `MEM_REQ_DEFERRED`.

A memory request may succeed, or it may fail because it was directed at an unsupported address ("wild" access), or because it was misaligned and the external memory system does not support misaligned addresses in requests, or some other reason. The first three response types handle these situations.

`MEM_REQ_DEFERRED` can be ignored for now; it is only relevant in Fife.

- The field `data` is a 64-bit field containing data in the case of a memory access that returns data from memory (FETCH, LOAD, AMO).
- The remaining fields are only for debugging.

Let's read some code: struct Fetch_to_Decode

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/Inter_Stage.bsv
Focus on:	struct Fetch_to_Decode

- The notation `Bit #(XLEN)` represents an XLEN-wide bit-vector. Parameterization by XLEN allows using the same code for both RV32 and RV64 implementations.
- In the `Fetch_to_Decode` struct, the only field of interest for the moment is `pc`, the value of the Program Counter. It is forwarded to later steps/stages because:
 - It is needed by AUIPC and Branch instructions to compute PC-relative values
 - It may be needed by JUMP instructions (JAL/JALR) to save a “return address”
 - In case of a trap or an interrupt, the PC needs to be saved in the MEPC CSR
- The remaining fields (`predicted_pc`, `epoch`, `inum`, `halt_sentinel`, ...) can be ignored for the moment—some are only needed in **Fife**, and some are only needed for debugger-control.

Let's read some code: `fn_Fetch`

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/Fn_Fetch.bsv
Focus on:	function <code>fn_Fetch</code>

- Conceptually: `fn_Fetch`: $PC \rightarrow (\text{Memory-request, Info-to-Decode})$
- Before the function definition is a definition of a struct `Result_F` representing the return-type of the function. It contains a pair of structs of type `Fetch_to_Decode` (to be sent to the Decode step/stage) and `Mem_Req` (to be sent to memory), respectively.

Note: Unlike C functions, which can only return values of size ≤ 64 bits, **BSV** functions can return arbitrarily-wide values, including complete structs, vectors, nested structs and vectors, etc..

- The return-type of the function `fn_Fetch` is `ActionValue #(Result_F)`. This specifies not only the type of the returned value (`Result_F`), but also that this function may have side-effects (`ActionValue #()`). This is useful both for user-documentation, but also for the compiler (side-effecting functions cannot be optimized as much as non-side-effecting ("pure") functions).
- Of the arguments, only `pc` is important, for now. `predicted_pc` and `epoch` are needed in **Fife**, and the remaining arguments are only for debugging.
- The function body is straightforward: it simply creates the `Fetch_to_Decode` and `Mem_Req` structs for the result. Note: the memory-request is a **LOAD** for a 4-byte value (a RISC-V instruction) at address `pc`.

The `zeroExtend` on the address extends 32 bits to 64 bits.

The "?" for the data field indicates we don't care what this value is (since it's a **LOAD**).

Let's read some code: `fn_Decode`

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	<code>Code/src_Common/Inter_Stage.bsv</code> <code>Code/src_Common/Fn_Decode.bsv</code>
Focus on:	<code>struct Decode_to_RR, function fn_Decode</code>

- Conceptually: `fn_Decode`: (Info-from-Fetch, Memory-response) → Info-to-Reg-Read (refer figure on Slide 7).
- This step/stage takes the incoming instruction and passes it along with some “classification” information. The classes are defined as values of the `enum OpClass` type (file `Inter_Stage.bsv`). Instructions of the `SYSTEM` and `FENCE` classes are sent directly to the Retire step/stage. `CONTROL`, `INT` and `MEM`-class instructions are sent into the corresponding fourth-stage execution paths shown in the figure on Slide 7.
- Struct `Decode_to_RR` (file `Inter_Stage.bsv`) represents the return-type of function `fn_Decode`. Some of the fields just pass through values that came in on the incoming `Fetch_to_Decode` struct.
- The `has_rs1`, `has_rs2` and `has_rd` are booleans indicating whether the instructing reads one or two register values, and whether it writes a register value. The `writes_mem` field is true if the instruction writes to memory (`STORE`, `SC`, `AMO` instructions). The `imm` field is the “immediate” value from the instruction (different kinds of instructions encode immediate values differently).
- The `exception` field is true if the Fetch memory-response returned a memory error, or if the instruction itself is not legal. In this case, the `cause` and `tval` fields provide additional information needed for the trap that will be taken in the Retire step/stage.
- The `fn_Decode` function is just a big nested conditional that examines the instruction and fills in these fields appropriately. It uses help-functions like `is_legal_LUI`, `is_legal_BRANCH`, and so on, defined in `Code/src_Common/Instr_Bits.bsv`.

Interlude

Badger: Ok, ok, enough of this pseudo-code, I think I get it, but when are we going to start writing the actual hardware code?

Owl: This isn't pseudo-code; this *is* the actual hardware code!

Let's read some code:

The Register File (General-Purpose Registers, or GPRs) (1/3)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	<code>Code/src_Common/GPRs.bsv</code>
Focus on:	<code>interface GPRS_IFC, module mkGPRs</code>

- The RISC-V ISA requires 32 GPRs, where GPR 0 always reads as 0.
- **BSV** designs are composed of *modules*, each of which has an *interface*. These are similar to *objects* and *methods* in object-oriented programming languages.
- `GPRs_IFC` defines an interface with several methods.

`read_rs1`: this method takes a 5-bit argument `rs1` which identifies a GPR, and returns an `xlen`-bit value which is the contents of that GPR. (And, similarly, `read_rs2`.)

`write_rd`: this method takes a 5-bit argument `rs1` which identifies a GPR, and an `xlen`-bit argument that is a value, and writes that value into that GPR. The “return type” `Action` indicates that it has a side-effect, and that it does not return any value.

- `read_dm`, `write_dm`: these methods can be ignored for now; they are intended for an optional RISC-V Debug Module to access registers.

Let's read some code:

The Register File (General-Purpose Registers, or GPRs) (2/3)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	<code>Code/src_Common/GPRs.bsv</code>
Focus on:	<code>interface GPRS_IFC, module mkGPRs</code>

- `mkGPRs` defines a module that presents the `GPRS_IFC` interfaces.
- The first line instantiates the **BSV** library module `mkRegFileFull`; `rf` names the interface to this instance. This module has methods `sub` to read a register and `.upd` to update a register with new value.
- The next section initializes all the registers to 0 when coming out of reset. This is not strictly required by the RISC-V ISA, but for debugging it is useful to emerge from reset in a known state.

We instantiate a standard **BSV** library register module `mkReg` that is initialized to zero, calling its interface `rg_init_index`.

The rule `rl_init` is effectively a loop, counting up on `rg_init_index`; at iteration j it writes 0 to `GPR[j]` using the `.upd` method for register files. The rule condition `("(! initialized)")` specifies when to terminate the loop.

`truncate()` converts the 6-bit `rg_init_index` value to 5 bits needed for indexes into the register file. **BSV** has very strict type-checking; there are no silent truncations and extensions; without this `truncate()` we'd get a type error.

- The rest of the module specifies implementations of each method.
"if (initialized)" is an *implicit condition* on each method, effectively blocking these methods from being invoked before complete initialization.

Let's read some code:

The Register File (General-Purpose Registers, or GPRs) (3/3)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/GPRs.bsv
Focus on:	interface GPRS_IFC, module mkGPRs

- The rest of the module specifies implementations of each method.
- “if (initialized)” is an *implicit condition* on each method, effectively blocking these methods from being invoked before complete initialization.
- Note that `write_rd` writes into the register file, but in `read_rs1/rs2`, if the index is zero, they ignore what's in the register file and return 0.

Alternatively, `write_rd` write 0 if the index is zero, and `read_rs1/rs2` could blindly return the register value.

- The *bsc* compiler cannot generate Verilog RTL for module `mkGPRs` because it is *polymorphic*—it has interface buses of variable width (`xlen`).

The module `mkGPRs_synth` fixes the variable `xlen` to be the constant `XLEN`. This module *can* be synthesized by *bsc*.

Let's read some code: `fn_Dispatch`

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	<code>Code/src_Common/Inter_Stage.bsv</code> <code>Code/src_Common/Fn_Dispatch.bsv</code>
Focus on:	structs <code>RR_to_Retire</code> , <code>RR_to_EX_Control</code> , <code>RR_to_EX</code> function <code>fn_Dispatch</code>

- Conceptually: `fn_Dispatch`: Info-from-Decode → Info to (`Retire`, `EX_Control`, `EX_Int`, `EX_DMem`) (refer figure on Slide 7).
The struct type `Result_Dispatch` contains four sub-structs for this purpose.
- Function `fn_Dispatch()` is straightforward, just constructing those four sub-structs and assembling them into the result struct. Each sub-struct is destined for one of the paths of execution shown in the figure.
- The `RR_to_Retire` struct is relevant for every instruction.
- The remaining three structs—`RR_to_EX_Control`, `RR_to_EX`, `RR_to_EX_DMem`—are each relevant only for a subset of instructions (`Control`, `Integer` and `Memory`, respectively), but this function fills them in blindly; those structs/fields will only be examined if relevant, so no harm is done if some of the values are bogus.

Let's read some code: `fn_EX_Control`

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	<code>Code/src_Common/Inter_Stage.bsv</code> <code>Code/src_Common/Fn_EX_Control.bsv</code>
Focus on:	<code>struct EX_Control_to_Retire</code> <code>function fn_EX_Control</code>

- Conceptually: `fn_EX_Control`: Info-from-Dispatch → Info to Retire (refer figure on Slide 7).
- Function `fn_Dispatch()` is straightforward, implementing the semantics of RISC-V BRANCH (conditional branch), JAL and JALR instructions.
- It also checks for misaligned targets.

Let's read some code: `fn_EX_Int`

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	<code>Code/src_Common/Inter_Stage.bsv</code> <code>Code/src_Common/Fn_EX_Int.bsv</code> <code>Code/src_Common/IALU.bsv</code>
Focus on:	<code>struct EX_to_Retire</code> <code>function fn_EX_Int</code> <code>function fn_IALU</code>

- Conceptually: `fn_EX_Int`: Info-from-Dispatch → Info to Retire (refer figure on Slide 7).
- Function `fn_Int()` is straightforward, implementing the semantics of RISC-V LUI, AUIPC and integer operations. The latter are packaged in function `fn_IALU`, which implements the semantics of RISC-V instructions:
 - (rs1, rs2) ADD, SUB, SLL, SLT, SLTU, SRL, SRA, AND, OR, XOR
 - (rs1, immed) ADDI, SLTI, SLTIU, ANDI, ORI, XORI, SLLI, SRLI, SRAI

Pause ...

All the types and functions described so far get compiled by the *bsc* compiler into Verilog.

Modulo minor syntactic differences, they look like types/functions written in many a popular software programming language.

They will be used, as-is, in Drum and Fife.

Drum: invoking the core pure functions from a sequential Finite State Machine (FSM)

This will give us a *hardware implementable* RISC-V CPU: **BSV** $\longrightarrow$ Verilog $\longrightarrow$ FPGA

- Not a very fast implementation, but still ≥ 1 *order of magnitude* faster than Bluesim or Verilog simulation (boot Linux in minutes instead of days).
- Can share the same system-surround (memory, devices) as Fife, and run the same software as Fife (because same CPU_IFC interface).
- More “obviously correct” than Fife: can be used as a verification reference (digital twin) for a more serious implementation such as Fife.
- More hospitable harness for debugging/verifying functional RISC-V parts (*i.e.*, the non-pipeline parts; the functions we’ve seen so far) than a pipelined CPU

Let's read some code: `exec_one_instr`

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	<code>Code/src_Drum/Drum_FSM.bsv</code>
Focus on:	<code>Stmt exec_one_instr</code>

- “`exec_one_instr`” is a compound “statement” (of type `Stmt`) and is a straightforward reading of the steps needed to execute one instruction:
Fetch → Decode → Register-read-and-dispatch → Execute-and-retire → Handle-exception-if-necessary
- Each step has been abstracted into an “action function” (such as `a_Fetch`) which we will study momentarily.
- The `seq-endseq` bracket indicate that its component actions should be performed sequentially (remember, **Drum** is not pipelined).
- The fourth action is itself an if-then-else corresponding to the four execution paths on Slide 7. Some of them use nested `seq-endseq` to sequence an execute action and a retire action.

Let's read some code: Execute instructions sequentially

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Drum/Drum.FSM.bsv
Focus on:	mk_AutoFSM

- “mk_AutoFSM” is a **BSV** library function that here embeds “exec_one_instr” into an infinite loop, thus fetching and executing instructions forever (the third arm of the if-then-else).
- Please ignore the first arm of the if-then-else: that is meant for debugger control.
- The second arm prioritizes taking an interrupt (if one is pending) over executing the next instruction. Thus, we always take interrupts *between* instructions.

Let's read some code: **Drum** CPU state

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Drum/CPU.bsv
Focus on:	module mkCPU

- The CPU state consists primarily of the Program Counter (`rg_pc`), the register file (`gprs`) and the Control-and-Status registers (`csrs`). (Please ignore the other registers for now.)
- Next, we see the registers holding inter-step values (`rg_Fetch_to_Decode` and so on).

This is a bit extravagant: in our sequential CPU, only one of these will be active at time, so we could optimize by merging them into a single register capable of holding the largest of the inter-stage values, at the expense of clarity.
- Next we see some instantiations of **BSV** library FIFO modules which hold memory requests and responses for Fetch (“IMem”) and for LOAD/STORE memory ops (“DMem”).

This may also seem a bit extravagant: in our sequential CPU, we are going to Fetch before LOAD/STORE and vice versa, so couldn't we share memory request/response FIFOs? Yes, but we want to keep our interface identical to Fife, where the pipeline may simultaneously Fetch and LOAD/STORE.
- Next we see some registers to hold information for exceptions and traps (`rg_exception`, `rg_epc`, `rg_cause`, `rg_tval`).
- The next section has registers for debugger control; please ignore this for now.

Let's read some code: **Drum** CPU Actions

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Drum/CPU.bsv
Focus on:	module mkCPU, section "Major CPU actions"

The bulk of the module body defines the major CPU action functions that we had invoked in the CPU FSM.

- Action `a_Fetch` invokes `fn_Fetch` (in file `src_Common/Fn_Fetch.bsv`), passing it the current PC (`rg_pc`).
The result `y` (of type `Result_F`, file `src_Common/Fn_Fetch.bsv`) contains two structs, a memory-request, and information for the Decode step. The former is placed in register `rg_Fetch_to_Decode`; the latter is enqueued on the FIFO `f_IMem_req`.
- The remaining functions are similar in flavor, each invoking some `fn_XXXX` function that we have already studied.
- Action `a_Retire_direct` handles `CSRxx`, `MRET`, `ECALL`, `EBREAK` instructions.
- Action `a_Retire_DMem` shows how we deal with memory exception responses, and responses with data of different sizes (Byte, Halfword, Word), with or without sign-extension.
- In the final "interface" section of the module, a line like:

```
interface fo_IMem_req = to_FIFOF_0 (f_IMem_req);
```

 simply exports the "output side" of FIFO `f_IMem_req` to the CPU interface (sub-interface of `CPU_IFC`).

Pause ...

... and we're done reading **Drum** code!

Modulo minor syntactic differences, it looks like a RISC-V simulator program you'd write in many a popular software programming language.

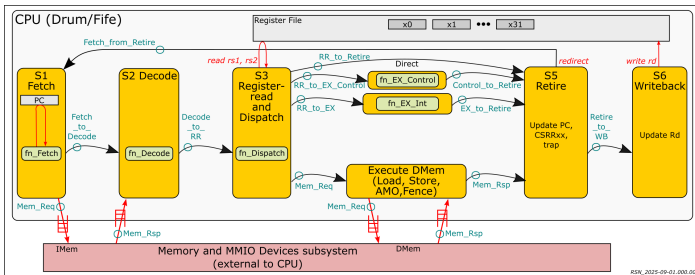
This code can be compiled to Verilog which, in turn, can be built into a Verilog simulation executable, or taken through FPGA or ASIC flows into real hardware.

And that's next on the agenda ...

Drum: Generating Verilog. Simulation.

The “CPU-surround” (shared by **Drum** and **Fife**)

Repository: https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
 Directory: Code/src_Top/
 Files: Top.bsv, Mems_Devices.bsv, supporting C files



- To make a simulation executable we need to connect the CPU interfaces to a CPU-surround (the purple box in the diagram).
- At a minimum, this should include a memory model
- Optional extras include a UART device (for console terminal I/O), a timer device (for measuring time and for timer interrupts), etc.
- This code is written in a mixture of **BSV** and C.

Let's build and run a **Drum** simulation, using Bluesim

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Directory:	Code/Build/Drum
Using:	Makefile (which mostly just "include"s ../Include.mk)

- Let's use the *bsc* compiler on the **BSV** sources:

```
$ make b.compile
bsc -u -sim ... misc bsc flags ...
-p ... directory path where bsc can find source files ...
.././src_Top/Top.bsv
```

... creates various intermediate files in the build_b/ directory.

- Let's use the *bsc* compiler to link the intermediate files:

```
$ make b.link
bsc -sim ... misc bsc flags ...
-e mkTop -o ./exe_Drum_RV32_bsim
... misc C files for the CPU-surround
```

... creates the executable ./exe_Drum_RV32_bsim

- Let's run the simulation on the famous "Hello World!" program (C, compiled to RISC-V binary):

```
$ make b.run_hello
ln -s -f .././Tools/Hello_World_Example_Code/hello.RV32.bare.memhex32 test.memhex32
./exe_Drum_RV32_bsim
... misc banner material ...
=====
Drum v0.81 2024-08-09 (FSM): starting execution at PC 80000000
=====
Hello World!
↑C
```

Let's build and run a **Drum** simulation, using Verilog (Verilator)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Directory:	Code/Build/Drum
Using:	Makefile (which mostly just "include"s ../Include.mk)

- Let's use the *bsc* compiler on the **BSV** sources to generate Verilog:

```
$ make v.compile
bsc -u -elab -verilog ... misc bsc flags ...
-p ... directory path where bsc can find source files ...
../src_Top/Top.bsv
```

... generates Verilog files in the `verilog/` directory.
You may wish to peruse this (later, at your convenience).

- Let's ask *bsc* to build a simulation executable (using Verilator):

```
$ make v.link
bsc -verilog -vsim verilator -use-dpi -vdir verilog ... misc bsc flags ...
-e mkTop -o ./exe_Drum.RV32_verilator
... misc C files for the CPU-surround
```

... creates the executable `./exe_Drum.RV32_verilator`

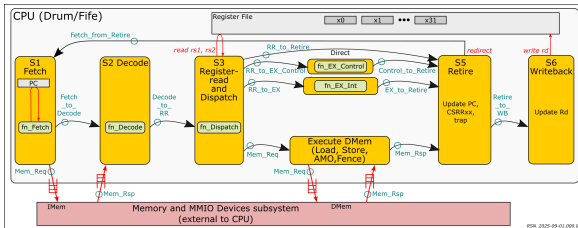
- Let's run the simulation on the famous "Hello World!" program (C, compiled to RISC-V binary):

```
$ make v.run_hello
ln -s -f ../Tools/Hello_World.Example_Code/hello.RV32.bare.memhex32 test.memhex32
./exe_Drum.RV32_verilator
... misc banner material ...
=====
Drum v0.81 2024-08-09 (FSM): starting execution at PC 80000000
=====
Hello World!
↑C
```

Fife: a six-stage pipelined implementation

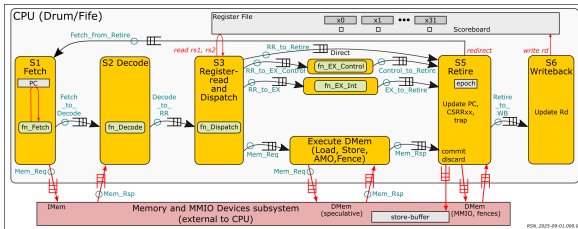
(invoking the same core pure functions as **Drum** that implement RISC-V semantics)

The Fife pipeline



In **Drum**, this figure represents the sequential states of an FSM, *i.e.*, the steps taken by an FSM, one after the other.

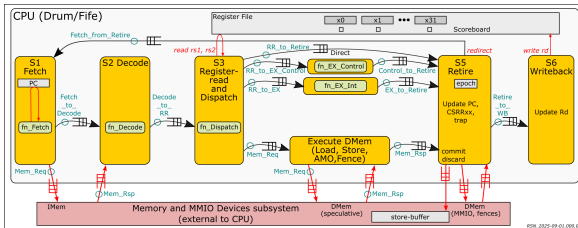
- One instruction is executed completely, going left to right through the figure, before we start again on the left, for the next instruction.



In **Fife**, this same figure represents a *pipeline*:

- Each module (Fetch, Decode, ...) is an infinite *process*, and they all run concurrently. Each is called a “pipeline stage”.
- All connections between modules are now FIFOs.
- Instructions “stream” through the figure, from left to right.
- At any given point in time, there could be several instructions “in flight”, in the various stages, in various states of completion.

The Fife pipeline: new concerns

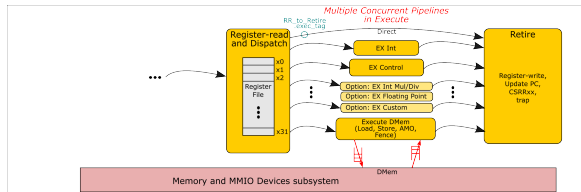


The **Fife** pipeline raises four new concerns:

- “In-order retirement”: the Execute pipelines (Stage 4, or S4) may operate at different speeds (number of clock cycles), so the order in which their results arrive at S5 may be different from the order in which they entered S4 from S3. But instructions must (logically) be performed in order.
- PC prediction to keep the pipeline full, and recovery from misprediction.
- Managing register “hazards”: correct ordering of reads and writes of GPRs for different instructions.
- Speculative memory STOREs: EX_DMem may execute a LOAD/STORE instruction I_n before we have completed instruction I_{n-k} that is logically ahead of it (in one of the other S4 pipelines). If I_{n-k} is discarded (misprediction, trap, interrupt), we should discard I_n , i.e., its STORE side-effect should not be performed.

We describe the solutions in the next few slides.

The Fife pipeline: In-order retirement



Issue: “In-order retirement”: the Execute pipelines (Stage 4, or S4) may operate at different speeds (number of clock cycles), so the order in which their results arrive at S5 may be different from the order in which they entered S4 from S3. But instructions must (logically) be performed in order.

Note that each of the S4 paths is FIFO-like (in order) even though their transit latencies may vary.

Solution:

- In Stage 3 (S3), for every instruction, we communicate a struct on the “Direct” path.
- We optionally *also* communicate a struct on one of the other paths (INT, Control, DMem) if it is that kind of instruction.
- In Stage 5 (S5), we use the struct at the head of the Direct path to decide whether or not to consume a struct from one of the other paths (INT, Control, DMem), and we retire the instruction.

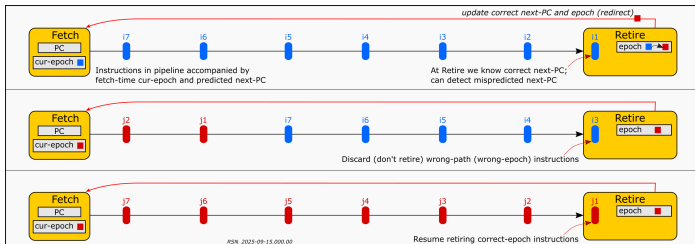
The **Fife** pipeline: PC prediction and recovery from misprediction

In Stage 1 (S1) ("Fetch"), when we issue a memory request for an instruction, the *only* information we have about it is the PC.

What PC should we use to fetch the next instruction, in order to keep the pipeline full? Normally the next PC is PC+4, but not if the fetched instruction turns out to be a (taken) BRANCH/JAL/JALR/ECALL/EBREAK, or if the instruction traps (memory fault on Fetch or LOAD/STORE, illegal instruction, misaligned BRANCH/JAL/JALR target), or if we take an interrupt, etc. These outcomes are only known in later stages.

When we finally know that an instruction's successor was mispredicted, how do we recover?

- In S1, we *predict* (guess) the next PC so we can do the next fetch. In **Fife** we blindly predict PC+4. *Much* better prediction is possible by analyzing the history of PCs; this is left for your future study.
- In Stage 5 (S5), we maintain an "epoch" register. When we know there was a misprediction, we increment the epoch, and send a struct to S1 indicating the correct PC and new epoch.
- S1 starts fetching from the correct PC. For each instruction, its epoch accompanies it through the pipeline. In S5, we discard all instructions of the wrong epoch, and start retiring again when we see the correct epoch.

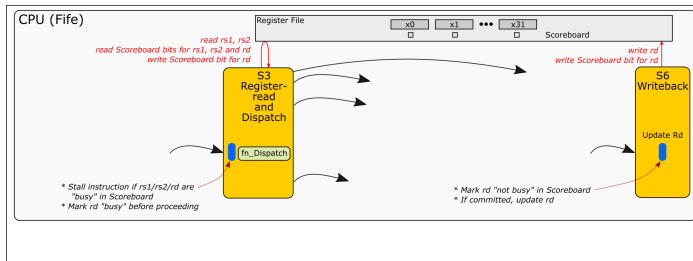


The Fife pipeline: Managing register “hazards”

Suppose an instruction I_n is in S4 or S5, and will write a result to GPR[x7].

Suppose a later instruction (say, I_{n+1}) tries to read from GPR[x7]. To avoid reading a stale value, it needs to wait (“stall”) until I_n has updated GPR[x7]. The danger of accessing the wrong GPR[j] value is called a “hazard”.

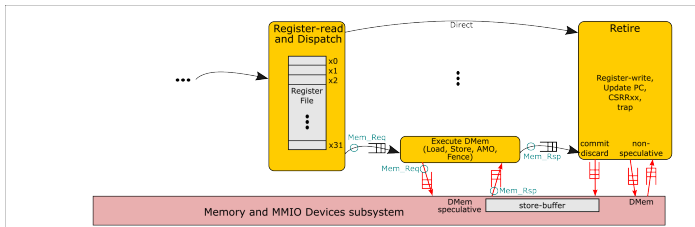
- Along with the GPRs, we maintain, for each GPR, a 1-bit “busy” value. The collection of 32 bits is called a “scoreboard”.
- When an instruction arrives at S3 we wait (stall), if necessary, until its registers are not busy. Then, we can read its Rs1 and Rs2 registers. If it has a destination register Rd, we mark it busy before allowing the instruction to move on to S4.
- In S5, when we retire an instruction, if it has an Rd, we send a struct back to S3 to free its busy bit (thereby possibly releasing an instruction might have been stalled waiting for this).



The Fife pipeline: Speculative memory LOADs and STOREs

EX_DMem may execute a LOAD/STORE instruction I_n before we have completed instruction I_{n-k} that is logically ahead of it (in one of the other S4 pipelines). If I_{n-k} is discarded (misprediction, trap, interrupt), we should discard I_n , i.e., its STORE side-effect should not be performed.

- In the Memory System (outside the CPU) we maintain a “Store Buffer”, where we keep an in-order record of any STORE memory updates.
- For a LOAD, we check the store buffer for the most recent update, if any, for the address of interest, and only do a usual memory access if there is no such record.
- In S5, when we retire this instruction, we send a “Commit/Discard” struct to the Store Buffer indicating how to dispose of its oldest entry: store it in memory, or discard.
- For MMIO addresses, even LOADs can have side-effects. We punt: the DMem.Speculative port does nothing but return the memory request with a DEFERRED indication, and we perform the operation in S5 once we know we're committing this instruction (using a small sequential FSM).



Let's read some code: **Fife's** S1 stage (fetch)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Fife/S1_Fetch.bsv
Focus on:	interface Fetch_IFC, module mkFetch, rl_Fetch_req

- In interface `Fetch_IFC` there are two outgoing FIFOs (to Decode and to Memory) and one incoming FIFO (PC redirection from S5).
- In module `mkFetch` we instantiate FIFOs for these interfaces.
- The rules `rl_Fetch_req` and `rl_Fetch_from_Retire` are infinite loops (processes). Each rule blocks (stalls/waits) until its rule-condition is true and all the implicit conditions of the methods it invokes are true; then it executes the rule body once; and this process repeats.
For example, the implicit condition of a FIFO `.enq` method is true only when there is space available in the FIFO (not full). The implicit condition of FIFO `.first` and `.deq` methods are true only when there is data available in the FIFO (non-empty).
- Rule `rl_Fetch_req` uses the current PC to invoke function `fn_Fetch()`, and enqueues its two results into the two outgoing FIFOs. It predicts the next PC as `PC+4` (this line can be replaced to use a more sophisticated predictor that knows about the history of past PCs).
- Rule `rl_Fetch_from_Retire` handles redirections (struct that was sent from S5). It updates the PC and the epoch registers, affecting future fetches by rule `rl_Fetch_req`.

Let's read some code: **Fife's** S3 stage (register read and dispatch)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Fife/S3_RR_S6_WB.bsv
Focus on:	interface RR_WB_IFC, module mkRR_WB

- In interface `RR_WB_IFC` there is one input FIFO (from Decode), one output FIFO for each of the paths through S4, and one input FIFO for register and scoreboard updates from S5.
- In module `mkFetch` we instantiate FIFOs for these interfaces.
- The rule `r1_RR_Dispatch` checks the scoreboard if the instruction's Rs1, Rs2 or Rd (if it has such register fields) are busy.
In the if-then-else that follows, if the scoreboard indicates we must stall, then we do nothing. This rule will try again, on the next clock
If the scoreboard permits us to proceed, we read the rs1 and rs2 registers and invoke `fn_Dispatch()` to produce the structs to be sent to the next stage.
Note that we blindly read GPRs rs1 and rs2, whether or not the instruction has such fields. There is no harm done if the instruction does not have such fields, since we only use them if relevant. Avoiding an unnecessary conditional will save us a MUX (multiplexor), which can be an expensive in hardware.
- Rule `r1_WB_from_Retire` accepts a struct sent by S5 and updates the scoreboard to “unbusy” a register (whether the S5-retired instruction was speculative or not), and updates the register if it was not speculative.

Fife's speculative LOADs and STOREs

- These are handled by the "Store_Buffer" in the memory system, which is outside the CPU.
- Being outside the CPU, and therefore not our focus today, we do not discuss this further here. In fact, we handle it in imported C code. If interested, please see the files:
 Code/src_Top/Mems_Devices.bsv
 Code/src_Top/C_Mems_Devices.c

Let's read some code: Interface for **Fife**'s S5 stage (retire/commit)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Fife/S5_Retire.bsv
Focus on:	interface Retire_IFC

- The interface Retire_IFC contains:
 - An input FIFO from S3: `fi_RR_to_Retire`
 - Input FIFOs from S4: `fi_EX_Control_to_Retire`, `fi_EX_Int_to_Retire` and `fi_DMem_S_rsp` (speculative DMem response)
 - An output FIFO `fo_DMem_S_commit` (speculative DMem commit/discard)
 - An output FIFO `fo_DMem_req` and input FIFO `fi_DMem_rsp` for MMIO (non-speculative) memory accesses
 - An output FIFO `fo_Fetch_from_Retire` to S1 for PC redirections
 - An output FIFO `fo_WB_from_Retire` to S3 for updating the GPRs and scoreboard.
- The remaining sub-interfaces are for a timer and timer interrupt, and debugger control; we do not have time to discuss these today.

Let's read some code: Module implementing **Fife's** S5 stage

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Fife/S5_Retire.bsv
Focus on:	... rules (see below) ...

- Function `fa_redirect_Fetch` is a help-function invoked from numerous rules to create a redirection struct and send it to S1 (enqueue on the outgoing FIFO to S1).
- Similarly, function `fa_update_rd` is a help-function invoked from numerous rules to create a GPR/scoreboard-update struct and send it to S3 (enqueue on the outgoing FIFO to S3).
- Bool `wrong_path` is the condition of whether the next incoming instruction from S4 is to be discarded (wrong epoch) or committed.
Rule `rl_Retire_wrong_path` performs the action of discarding wrong-path instructions including, if necessary, releasing any busy register in the scoreboard (message to S3) and discarding any pending STORE in the Store Buffer.
- Rule `rl_Retire_Direct_exception` retires any exception that was discovered in stages S2-S3 (including fetch memory-access errors and illegal instructions).
- Rules `rl_Retire_EX_Control` and `rl_Retire_EX_Int` retire BRANCH, JAL, JALR and integer ALU instructions that came through the EX_Control and EX_Int S4 paths.
- Rule `rl_Retire_EX_DMem` commits LOAD/STORE instructions that were performed speculatively (*i.e.*, not DEFERRED).
- Rule `rl_Retire_DMem_deferred` handles DEFERRED memory instructions, now issuing the memory request. Rule `rl_Retire_DMem_rsp` handles the response.
- In any of the preceding rules, if there was an exception (trap or interrupt), they set up rule `rl_exception` to handle it.

Pause ...

... and we're done reading **Fife** code!

Congratulations!

The key difference from a traditional programming language is the execution semantics based on *concurrent rules*, a.k.a, *Guarded Atomic Actions*, as opposed to traditional sequential statements.

This code can be compiled to Verilog which, in turn, can be built into a Verilog simulation executable, or taken through FPGA or ASIC flows into real hardware.

And that's next on the agenda ...

Fife: Generating Verilog. Simulation.

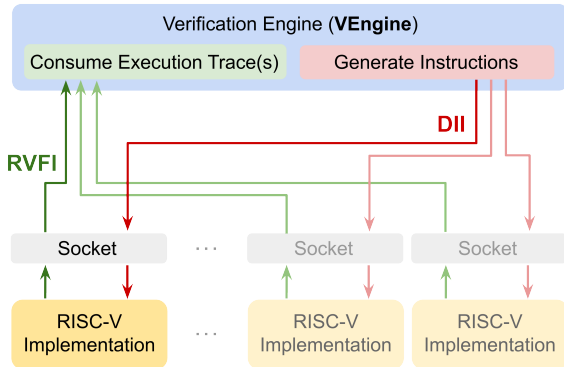
Please refer back to Section 3 (page 28) for the steps to build and run a **Drum** simulation.

The steps for **Fife** simulation are exactly the same, just substitute “Fife” wherever you see “Drum” in the previous instructions.

Verification using test programs

Verification using **TestRIG**

About TestRIG



TestRIG is a system for automated randomized testing of RISC-V implementations:

Joannou, A., Rugg, P., Woodruff, J., Fuchs, F.A., van der Maas, M., Naylor, M., Roe, M., Watson, R.N.M., Neumann, P.G. and Moore, S.W., *Randomized testing of RISC-V CPUs using direct instruction injection*, IEEE Design & Test 41:1, February 2024, pp.40-49, DOI 10.1109/MDAT.2023.3262741

TestRIG is free and open-source software, available at:
<https://github.com/CTSRD-CHERI/TestRIG>

The VEngine generates randomized sequence of instructions.

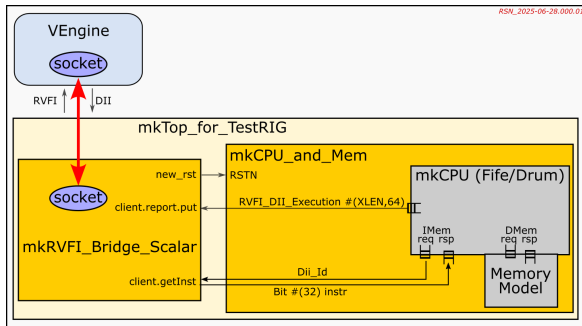
These instruction sequences are fed into two RISC-V implementations (A and B) using *Direct Instruction Injection* (DII). Typically,

- Implementation “A” is a RISC-V “reference model”. Here we use a simulator built from the *official Sail RISC-V Formal Specification* (<https://github.com/riscv/sail-riscv>).
- Implementation “B” is the “Design Under Test” (or DUT). Here, it is a **Fife** simulation.

Both A and B report the outcomes of *each* instruction to VEngine, which compares them and reports any discrepancy.

Bonus: A failing instruction sequence could be hundreds/thousands of instructions long. The VEngine will automatically try to *shrink* these sequences, often down to just a handful of instructions, making diagnosis of the error *much* easier.

Fitting **Fife** (or any RISC-V CPU design) into **TestRIG**



- mkCPU is instrumented to collect the information for the RVFI_DII_Execution struct. These structs are streamed out on a new FIFO output interface `fo_rvfi_reports` added to the CPU_IFC interface.

- We create a new module `mkCPU_and_Mem` (file `TestRIG/src_Top_TestRIG/CPU_and_Mem.bsv`).

Unlike the normal `Top.bsv` for standalone simulation, here **we do not connect the IMem memory interfaces (for Fetch) to the memory model**.

Instead, we have logic that, for every IMem request, consumes an instruction from **TestRIG** and feeds that in the IMem response (we ignore the instruction address (PC) in the request).

- mkRVFI_Bridge_Scalar is from U.Cambridge GitHub: <https://github.com/CTSRD-CHERI/BSV-RVFI-DII.git>. We obtain a local copy (see `TestRIG/vendor/`).
- We create a new top-level BSV module `mkTop_for_TestRIG` (file `TestRIG/src_Top_TestRIG/Top_TestRIG.bsv`) that instantiates `mkRVFI_Bridge_Scalar` and `mkCPU_and_Mem`, and connects up the reset, report and `getInst` interfaces.

And that's it! We're ready to roll!

Let's read some code: instrumenting the CPU for RVFI reports

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	<code>Code/vendor/RVFI_DII.Types/RVFI_DII.Types.bsv</code> <code>Code/src_Common/RVFI_Reports.bsv</code> <code>Code/src_Fife/S5_Retire.bsv</code>
Focus on:	... (see below) ...

- File `RVFI_DII.Types.bsv` defines the struct type `RVFI_DII.Execution` for the RVFI “Reports” sent by the CPU to VEngine for each instruction. For each kind (opcode) of instruction, only some fields will be relevant.
- Module `mkRVFI_Report` (file `Code/src_Common/RVFI_Reports.bsv`) contains “help” methods for producing the RVFI report for each kind of instruction. They’re all very similar; take a look at `rvfi_Int` for example.
- Module `mkRetire` (file `Code/src_Fife/S5_Retire.bsv`) instantiates module `mkRVFI_Report`.
- In module `mkRetire` (file `Code/src_Fife/S5_Retire.bsv`), look at rule `rl_Retire_EX_Int` and observe the invocation of `rvfi_report.rvfi_Int()` to create the RVFI report for integer (ALU) instructions.

Let's read some code: `mkCPU_and_Mem`

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	<code>TestRIG/src_Top_TestRIG/CPU_and_Mem.bsv</code>
Focus on:	... (see below) ...

- Interface `CPU_and_Mem_IFC` has the three sub-interfaces shown in the diagram on an earlier slide.
- Module `mkCPU_and_Mem` instantiates:
 - `mkCPU` (modified only with instrumentation to produce `RVFI_DII_Execution` reports for each instruction).
 - `mkMems_Devices` (unmodified). Note that we pass “dummy” FIFO interfaces for `IMem_reqs` and `IMem_rsps`, where previously we would have passed in the corresponding sub-interfaces from `mkCPU`.
- Module `mkCPU_and_Mem`'s rule `rl_step1`, which is executed during startup, invokes `mems_devices.init(init_params)`, where we have supplied the memory base address and memory size expected by **TestRIG**.
- Module `mkCPU_and_Mem`'s rule `rl_relay_instrs` gets a memory Fetch request from the CPU, consumes an instruction from `VEngine` (the `f_instrs` FIFO), and sends a memory Fetch response to the CPU.
- Module `mkCPU_and_Mem`'s rule `rl_relay_rvfi` relays the `RVFI_DII_Execution` reports from the CPU out to `mkRVFI_Bridge_Scalar`
- **TODO: explain how we deal with “wrong-path” Fetch requests.**

Let's read some code: mkTop_for_TestRIG

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	TestRIG/src_Top_TestRIG/Top_TestRIG.bsv
Focus on:	... (see below) ...

The module `mkTop_TestRIG` is the top-level (Empty interface) of the simulation executable for running with TestRIG

- It instantiates modules `mkRVFI_DII_Bridge_Scalar` and `mkCPU_and_Mem`. In the latter, “reset_by bridge.new_rst” phrase allows it to be reset by the former (this is invoked after each test run by **TestRIG** so that each test begins with the CPU in a known, predictable state).
- The two rules merely forward the DII instruction traffic from VEngine to CPU, and the RVFI_DII_Execution reports from the CPU to the VEngine.

Demo: **TestRIG** comparing Sail RISC-V Formal Spec and **Fife**

Start the DUT simulation (**Fife** with its **TestRIG** wrapper) in one window.

Start **TestRIG** in another window.

- Study and discuss command-line arguments for **TestRIG**
- Note that the “A” comparand is automatically started by **TestRIG**, and is an executable created from the Sail RISC-V Formal Specification.
The “B” comparand (**Fife**) could also be automatically started by **TestRIG**, but we have started it manually here just for demo purposes.
- Study and discuss **TestRIG**’s output.
- Deliberately introduce a bug in **Fife**, and repeat a run; study and discuss **TestRIG**’s “shrinking” and discrepancy-reporting.

Comparing **TestRIG** with other verification methodologies

Compare: Random program generation (<https://github.com/chipsalliance/riscv-dv>)

- Easier random generation by bypassing the instruction-fetch mechanism (no need to generate “control-flow-correct” random programs).
- Shrinkage of tests to “minimal” counter-examples.
- Very powerful generator mechanisms (Haskell QuickCheck library).

Compare: ad-hoc test programs:

- *Much* larger repertoire of tests.
- *Many* more corner cases explored.
- Easier random generation by bypassing the instruction-fetch mechanism (no need to generate “control-flow-correct” random programs).
- Shrinkage of tests to “minimal” counter-examples.

Compare: Formal Verification tools (e.g., Jasper):

- **TestRIG** is automatically checking a gazillion correctness properties: each test is checking the property that Fife has the same behavior as Sail. With Formal Verification tools, users have to write all properties by hand.
- *Many* more corner cases are likely explored.

Building and running on FPGA

... TBD ...

Links to resources (all free and open-source):

bsc compiler for BSV and BH, libraries, discussion forums:

<https://github.com/B-Lang-org/bsc>
<https://groups.io/g/b-lang-discuss>
<https://groups.io/g/b-lang-announce/topics>
<https://github.com/B-Lang-org/bsc-contrib> (incl. AMBA/AXI fabrics)
<https://github.com/BSVLang/Main>

Textbook, lecture slides, exercises, full code for **Drum** and **Fife** RISC-V CPUs (buildable, runnable):

https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design

RISC-V CPUs (all RV64GC, Linux-capable, coded in BSV):

Piccolo (3-stage)	https://github.com/bluespec/Piccolo	
Flute (5-stage)	https://github.com/bluespec/Flute	
Toooba (Superscalar, out-of-order)	https://github.com/bluespec/Toooba	
CHERI (RISC-V + security capabilities):	https://github.com/CTSRD-CHERI	(U.Cambridge)

RISC-V IP (coded in BSV):

CLINT (timer and sw interrupts)	https://github.com/bluespec/CLINT
PLIC (interrupt controller)	https://github.com/bluespec/PLIC
Debug Module	https://github.com/bluespec/RISCV_Debug_Module
gdbstub	https://github.com/bluespec/RISCV_gdbstub

End

Q&A

Slides preparation tools:

- Text and layout: LaTeX with Beamer package, Madrid theme
- Diagrams: Inkscape in SVG, export to PNG